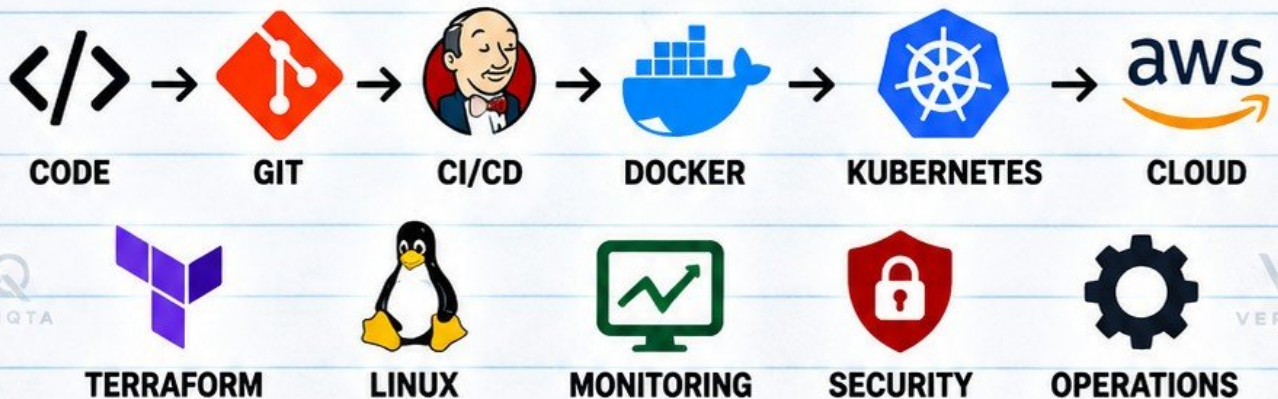


Complete DevOps Bootcamp

20-PAGE BEGINNER TO PRACTITIONER GUIDE



- ✓ Clear Explanations
- ✓ Real-World Examples
- ✓ Practical Commands
- ✓ Hands-On Learning
- ✓ Production-Focused
- ✓ Build a Complete Project

*Same
Concepts
Used in Real
Companies*

*Practice
Build
Get Job Ready*

*Your DevOps Journey
Starts Here!*

**A MORE
PRACTICAL YOU**

DevOps: The Complete Picture

PEOPLE + PROCESS + TOOLS = FASTER, BETTER SOFTWARE

WHAT IS DEVOPS?

DevOps is a way of working that brings **Development (Dev)** and **Operations (Ops)** together to build, test, deploy and operate software faster, more reliably and with continuous improvement.

WHY DOES IT EXIST?

- Faster delivery of value
- More reliable systems
- Better collaboration
- Automation of repetitive work
- Quick response to problems
- Continuous improvement
- Happier customers

DEVELOPMENT (DEV)



- Write code
- Build features
- Run tests
- Commit changes
- Work in short cycles

Goal: Build the right thing



WORK
TOGETHER

SHARED GOALS
SHARED OWNERSHIP
SHARED SUCCESS

OPERATIONS (OPS)



- Deploy applications
- Keep systems running
- Monitor and troubleshoot
- Manage infrastructure
- Ensure security and reliability

Goal: Keep it running reliably

THE DEVOPS LIFECYCLE (CONTINUOUS FLOW)

IT STARTS
WITH AN IDEA



Turn ideas
into working
software.



IT ENDS WITH
HAPPY USERS



Deliver value
to customers
continuously.

REAL-WORLD EXAMPLE

You push code → it is automatically tested, built, deployed to production, monitored, and if something goes wrong, you fix it quickly.

DEVOPS CONNECTS EVERYTHING

DevOps connects people, processes and tools across development, operations, security, and the business to deliver value continuously.

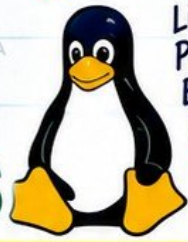
REMEMBER THIS

DevOps is not a tool. It is a mindset, a culture and a better way to build and run software.



Linux

for DevOps Engineers



Linux Powers Everything in DevOps

FILESYSTEM | USERS | PERMISSIONS | PROCESSES | SERVICES | PACKAGES | LOGS | SSH | ESSENTIAL COMMANDS

1. WHAT IS LINUX?

- Linux is an open-source operating system used to run servers, cloud instances, containers and DevOps tools.
- It is stable, secure, flexible and widely used in production.
- Most DevOps work involves using Linux on servers, in the cloud or in containers.

2. LINUX FILESYSTEM

- Linux uses a single root directory called / (root).
- Everything is a file or directory.

Common Directories

/	Root directory
/home	User home directories
/etc	Configuration files
/var	Logs and variable data
/usr	Applications and tools
/opt	Optional software
/tmp	Temporary files

3. USERS AND GROUPS

- Users run commands and own files.
- Groups help manage permissions.
- Use least privilege (not always root).

Useful Commands

whoami	Show current user
id	Show user and groups
users	List logged in users
sudo	Run a command as root
adduser	Create a new user

4. PERMISSIONS

- Permissions control who can read, write or execute files.
- Three types: user (u), group (g), others (o).
- Use numbers (chmod) or symbols (rwx).

Common Commands

ls -l	Show file permissions
chmod 755 file	Change permissions
chown user:file	Change ownership

5. PROCESSES

- Processes are running programs.
- You can view, manage and stop them.

Useful Commands

ps aux	List all processes
top	Real-time process view
htop	Interactive process view
kill PID	Stop a process
killall name	Stop by process name

6. SERVICES

- Services run in the background (e.g. web servers, databases).
- Use **systemctl** to manage services.

Useful Commands

systemctl status nginx	Check status
systemctl start nginx	Start service
systemctl stop nginx	Stop service
systemctl restart nginx	Restart service
systemctl enable nginx	Start on boot

7. PACKAGES

- Package managers install, update and remove software.
- Common package managers: apt (Ubuntu/Debian) and yum/dnf (CentOS/RHEL).

Useful Commands (Ubuntu/Debian)

sudo apt update	Update package list
sudo apt install nginx	Install a package
sudo apt remove nginx	Remove a package

8. LOGS

- Logs help you understand what happened on a system.
- Application logs are usually in **/var/log**.
- Use **journalctl** for system logs (on systems with systemd).

Useful Commands

journalctl -xe	View system logs
tail -f /var/log/syslog	Follow a log file
tail -n 50 app.log	Show last 50 lines
grep error app.log	Search for errors

9. SSH (SECURE SHELL)

- SSH lets you securely connect to remote Linux servers.
- Use SSH keys instead of passwords.
- Keep your private key safe.

Useful Commands

ssh user@server-ip	Connect to a server
ssh -i key.pem user@ip	Use a key file
scp file.txt user@ip:/path	Copy a file
ssh-keygen	Generate a key pair

10. ESSENTIAL LINUX COMMANDS

- These commands are used every day in DevOps work.

pwd	Show current directory	cp src dest	Copy files
ls -la	List files in detail	mv src dest	Move or rename files
cd /path	Change directory	cat file	View file content
mkdir dir	Create a directory	grep text file	Search for text
rm file	Remove a file	df -h	Check disk space
		free -h	Check memory usage

PRACTICE THIS

- Explore the filesystem
- Create a user and set permissions
- Install a package (nginx)
- Start and check a service
- View logs and find an error

PRACTICE TODAY
A STRONGER YOU TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



You Can Do This!

03 | Linux Troubleshooting

Find the Problem | Get Evidence | Apply the Fix | Verify the Result

1. PROCESS CHECKS

View and manage running processes.

Command	What it does
<code>ps aux</code>	List all processes
<code>top</code>	Real-time process view
<code>htop</code>	Interactive process view (if installed)
<code>ps -ef grep nginx</code>	Find a specific process
<code>kill PID</code>	Stop a process (use with care)
<code>kill -9 PID</code>	Force stop a process (last resort)

2. DISK SPACE

Check disk usage and find large files.

Command	What it does
<code>df -h</code>	Show disk space usage
<code>du -sh /path</code>	Show size of a directory
<code>du -h --max-depth=1</code>	Find large directories
<code>find / -type f -size +100M</code>	Find files larger than 100MB

3. MEMORY USAGE

Check memory and identify issues.

Command	What it does
<code>free -h</code>	Show memory usage
<code>top</code>	Check memory in real time
<code>htop</code>	Interactive view (if installed)
<code>cat /proc/meminfo</code>	Detailed memory information

4. NETWORK AND PORTS

Check open ports and network connections.

Command	What it does
<code>ss -tln</code>	Show listening ports
<code>ss -tulpn</code>	Show ports with process names
<code>netstat -tln</code>	Show listening ports (older tool)
<code>lsof -i :80</code>	Find process using port 80
<code>ping google.com</code>	Test network connectivity
<code>curl -I https://example.com</code>	Test HTTP/HTTPS connectivity

5. SERVICES (SYSTEMD)

Check and manage services.

Command	What it does
<code>systemctl status nginx</code>	Check service status
<code>systemctl start nginx</code>	Start a service
<code>systemctl stop nginx</code>	Stop a service
<code>systemctl restart nginx</code>	Restart a service
<code>systemctl enable nginx</code>	Start on boot
<code>systemctl disable nginx</code>	Disable on boot
<code>systemctl list-units --type=service</code>	List all services

6. LOGS (JOURNALCTL)

View system and service logs.

Command	What it does
<code>journalctl -xe</code>	Show recent system logs (with errors)
<code>journalctl -u nginx</code>	View logs for a service
<code>journalctl -u nginx -f</code>	Follow logs in real time
<code>journalctl --since "1 hour ago"</code>	Logs from last hour
<code>tail -f /var/log/syslog</code>	Follow system log
<code>tail -f /var/log/nginx/access.log</code>	Follow application log

7. FILE PERMISSIONS

Check and fix permission issues.

Command	What it does
<code>ls -l</code>	Show file permissions
<code>chmod 755 file</code>	Change permissions
<code>chown user:group file</code>	Change owner and group
<code>id</code>	Show current user and groups
<code>sudo -l</code>	Show sudo permissions
<code>stat file</code>	Detailed file information

8. COMMON ISSUES AND FIXES

Issue / Error	What to Check
Permission denied	File permissions, user, sudo
Command not found	Package installed? PATH?
Port already in use	Use ss or lsof to find process
Service not starting	Check logs with journalctl
Disk full	Use df -h and find large files
High memory usage	Use free, top, htop
Application not reachable	Check service, ports, firewall
Process keeps crashing	Check logs and configuration

9. TROUBLESHOOTING FLOW

- 1 → Understand the problem (What is not working?)
- 2 → Check symptoms (Errors, logs, status, metrics)
- 3 → Gather evidence (Use the right commands)
- 4 → Identify the root cause
- 5 → Apply the fix
- 6 → Verify the result (Did it actually work?)

10. PRACTICE THIS

- ✓ Find the top 5 processes using memory.
- ✓ Find which process is using port 80.
- ✓ Check disk usage and find the largest directory.
- ✓ View and follow nginx logs.
- ✓ Restart a service and verify it is running.
- ✓ Create a test file, change its permissions, and verify.



JUNIOR ENGINEER TIP

Logs are your best friend. They tell you what actually happened. Always check logs before making changes.



REMEMBER

- Don't randomly kill processes.
- Don't delete logs.
- Check evidence before making changes.
- Use the right tool for the problem.
- Verify the fix.

Networking for DevOps

IP ADDRESSES | DNS | PORTS | TCP/UDP | HTTP/HTTPS | ROUTING | FIREWALLS | NAT

1. IP ADDRESSES

- An IP address is a unique number that identifies a device on a network.
- IPv4 example: **192.168.1.10**
- IPv6 example: **2001:db8::1**
- Used for communication between devices, servers and services.

Useful Commands

ip a Show IP addresses
hostname -I Show system IP
ip route Show routing table

2. DNS (DOMAIN NAME SYSTEM)

- DNS converts human-readable names (example.com) to IP addresses.
- It helps you access services using names instead of numbers.
- Example: **example.com** → **93.184.216.34**

Useful Commands

nslookup example.com Query DNS
dig example.com Detailed lookup
host example.com Simple lookup

3. PORTS

- Ports identify which application should receive the traffic.
- Well-known ports:
 - 22 - SSH
 - 80 - HTTP
 - 443 - HTTPS
 - 3306 - MySQL
 - 5432 - PostgreSQL

Useful Command

ss -tln Show open ports (preferred)
netstat -tln Show open ports (older tool)

4. TCP vs UDP

- TCP (Transmission Control Protocol)**
 - Reliable, ordered, connection-based
 - Used for HTTP, HTTPS, SSH, databases
- UDP (User Datagram Protocol)**
 - Faster, no guaranteed delivery
 - Used for DNS, video, streaming, gaming

5. HTTP vs HTTPS

- HTTP:** transfers data in plain text (not encrypted).
- HTTPS:** encrypts data using TLS (Transport Layer Security).
- HTTPS protects sensitive data such as logins, tokens and personal information.

6. ROUTING

- Routing decides where to send network traffic to reach the destination.
- Routers use routing tables to forward packets between networks.

Commands

ip route Show routing table
traceroute example.com Trace the path

7. HOW AN APPLICATION REQUEST TRAVELS



8. FIREWALLS

- A firewall controls which traffic is allowed in or out.
- It blocks unwanted traffic and allows only trusted traffic.
- Used in servers, cloud environments and Kubernetes clusters.

Commands (Linux - UFW)

sudo ufw status Check firewall status
sudo ufw allow 22 Allow SSH
sudo ufw deny 80 Block HTTP

9. NAT (NETWORK ADDRESS TRANSLATION)

- NAT converts private IP addresses (e.g. 192.168.1.10) to a public IP address for internet access.
- Used in home networks, cloud VPCs and Kubernetes.
- It allows multiple devices to share one public IP.

Private vs Public IP

Private IP: 192.168.1.10 (internal use)
Public IP: 203.0.113.10 (internet facing)

10. PRACTICE THIS

- ✓ Check your IP address.
- ✓ Look up a domain using dig or nslookup.
- ✓ Check open ports on your system.
- ✓ Use traceroute to see the path to a website.
- ✓ Test HTTP vs HTTPS in your browser.
- ✓ Check your firewall status.
- ✓ Explore how NAT works in your home or cloud environment.



REMEMBER

- Networks make DevOps possible.
- Understand the basics, it helps you debug faster in real environments.
- Everything is connected.

Git and GitHub Workflow

Repository → Branch → Change → Commit → Push → Pull Request → Review → Merge

1. WHAT IS GIT?

- Git is a version control system that tracks changes in your code.
- It helps you work safely, go back in time, and collaborate with others.
- It runs locally on your computer.

2. WHAT IS GITHUB?

- GitHub is a cloud platform for Git repositories.
- It helps teams collaborate, review changes, and manage code.
- It adds features like pull requests, issues, and project management.

3. KEY TERMINOLOGY

Term	Meaning
Repository (repo)	Project folder with full history
Branch	Separate line of development
Commit	Snapshot of changes
Push	Upload changes to remote repo
Pull Request (PR)	Request to merge changes
Merge	Combine changes into main branch

4. THE GIT & GITHUB WORKFLOW



5. ESSENTIAL GIT COMMANDS

Command	What it does
<code>git init</code>	Create a new repository
<code>git clone <url></code>	Clone a remote repository
<code>git status</code>	Show current status
<code>git add <file></code>	Stage changes
<code>git commit -m "message"</code>	Save changes with a message
<code>git branch</code>	List branches
<code>git checkout -b <name></code>	Create and switch to a branch
<code>git push origin <branch></code>	Push changes to GitHub
<code>git pull</code>	Get latest changes
<code>git log</code>	View commit history

6. USEFUL GITHUB ACTIONS

Action	What it does
Create repository	Start a new project
Push code	Upload your code
Open pull request	Request to merge changes
Review changes	Discuss and approve
Merge pull request	Combine into main branch
Issues	Track tasks and bugs
Projects	Manage work
Actions (CI/CD)	Automate tests and builds

JUNIOR ENGINEER TIP
 Make small, focused commits with clear messages. It makes reviews easier and your history clean.

7. MERGE CONFLICTS

- A merge conflict happens when two people change the same part of a file.
- Git cannot automatically decide which change to keep.
- You must manually resolve it.

Steps to Resolve a Conflict

- 1 Identify the conflicting file
- 2 Open the file and review changes
- 3 Edit and keep the correct content
- 4 Stage the file (`git add <file>`)
- 5 Commit the resolution
- 6 Push the changes

8. PRACTICAL EXAMPLE

```

git clone https://github.com/user/repo.git # Clone repo
cd repo # Go into folder
git checkout -b feature/login # Create branch
# make code changes
git add . # Stage changes
git commit -m "Add login feature" # Commit
git push origin feature/login # Push to GitHub
# Open a pull request on GitHub
  
```

9. COMMON MISTAKES

- ✗ Committing directly to main
- ✗ Not pulling before pushing
- ✗ Large commits with unclear messages
- ✗ Ignoring merge conflicts
- ✗ Pushing sensitive information
- ✗ Not using branches
- ✗ Not writing meaningful commit messages

10. REMEMBER

- ✓ Use branches for new features
- ✓ Keep commits small and clear
- ✓ Pull the latest changes regularly
- ✓ Use pull requests for collaboration
- ✓ Resolve conflicts carefully
- ✓ Never commit secrets
- ✓ A clean Git history shows professionalism

PRACTICE TODAY
BUILD TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



VERIQTA

Code
Collaborate
Create Opportunities

Bash and Automation

Variables | Pipes | Redirection | Exit Codes | Environment Variables | Scripts | Loops | Conditions

1. SHELL BASICS

The Bash shell lets you run commands, write scripts and automate tasks.

Command	What it does
<code>echo "Hello"</code>	Print text
<code>pwd</code>	Show current directory
<code>ls -la</code>	List files in detail
<code>cd /path</code>	Change directory
<code>whoami</code>	Show current user
<code>date</code>	Show date and time
<code>clear</code>	Clear the terminal

2. VARIABLES

Variables store data for later use.

Example	What it does
<code>name="DevOps"</code>	Create variable
<code>echo \$name</code>	Use variable
<code>read user</code>	Take user input
<code>ENV=value</code>	Set environment variable
<code>echo \$HOME</code>	Show environment variable

Tips

- Use `$` before variable name
- Use quotes for values with spaces
- Use environment variables for configuration

3. PIPES

Pipes (`|`) send the output of one command to another command.

Command	What it does
<code>ls -l less</code>	View output page by page
<code>ps aux grep nginx</code>	Filter output
<code>cat file sort</code>	Sort content
<code>cat file uniq</code>	Remove duplicates
<code>cat file wc -l</code>	Count lines
<code>df -h grep /dev</code>	Filter disk info

Pipes help you chain commands together to solve problems.

4. REDIRECTION

Redirect input and output to files.

Command	What it does
<code>></code>	Overwrite output to a file
<code>>></code>	Append output to a file
<code><</code>	Take input from a file
<code>2></code>	Redirect error output
<code>&></code>	Redirect both output and errors
<code>command > file</code>	Save output to file
<code>command >> file</code>	Append output to file
<code>command < file</code>	Use file as input

5. EXIT CODES

Every command returns an exit code.

Code	Meaning
<code>0</code>	Success
<code>1</code>	General error
<code>2</code>	Misuse of command
<code>126</code>	Permission denied
<code>127</code>	Command not found

Check Exit Code

`echo $?` Show last command exit code
`command && echo "Success"`
`command || echo "Failed"`

6. SCRIPTS

Write scripts to automate repetitive work.

Example	What it does
<code>#!/bin/bash</code>	Start of a script
<code>echo "Hello"</code>	Print text
<code>name=\$1</code>	Take first argument
<code>echo "Name: \$name"</code>	Use argument
<code>chmod +x script.sh</code>	Make script executable
<code>./script.sh</code>	Run the script

Tips

- Use meaningful names
- Add comments (`#`)
- Make scripts executable

7. LOOPS

Loops repeat commands.

Example	What it does
<code>for i in 1 2 3; do echo \$i; done</code>	Loop through a list
<code>for file in *.log; do echo \$file; done</code>	Loop through files
<code>while true; do echo "Running"; done</code>	Infinite loop (use with care)

8. CONDITIONS

Conditions let you make decisions.

Example	What it does
<code>if [-f file]; then ... fi</code>	Check if file exists
<code>if [-d dir]; then ... fi</code>	Check if directory exists
<code>if [\$1 = "prod"]; then ... fi</code>	Check a value
<code>if command; then ... fi</code>	Check command success
<code>if [\$a -gt 10]; then ... fi</code>	Numeric comparison

9. PRACTICAL EXAMPLES

Real examples you will use in DevOps.

Command	What it does
<code>for i in {1..5}; do echo \$i; done</code>	Loop 1 to 5
<code>cat access.log grep "404" wc -l</code>	Count 404 errors
<code>find /var/log -name "*.log"</code>	Find log files
<code>tar -czf backup.tar.gz /data</code>	Create a backup
<code>if systemctl is-active nginx; then echo "Nginx is running"; fi</code>	Check service status
<code>curl -s https://example.com > /dev/null && echo "Site OK" echo "Site Down"</code>	Check website

10. AUTOMATING REPETITIVE WORK

- ✓ Write scripts for common tasks.
- ✓ Use loops to process multiple files.
- ✓ Use cron jobs to schedule tasks.
- ✓ Combine commands with pipes and redirection.
- ✓ Always check exit codes.
- ✓ Log your script output.



JUNIOR ENGINEER TIP

Start small. Automate one repetitive task today. Small scripts save time, reduce errors and make you a better engineer.



COMMON MISTAKES

- ✗ Forgetting to use quotes.
- ✗ Not checking exit codes.
- ✗ Hardcoding sensitive information.
- ✗ Not handling errors.
- ✗ Writing complex scripts without testing.
- ✗ Giving incorrect file permissions.

CI/CD Fundamentals

Code → Build → Test → Scan → Package → Deploy → Verify → Rollback

1. WHAT IS CI/CD?

- CI/CD is a set of practices to automate the software delivery process.
- CI = Continuous Integration (build, test, scan, package).
- CD = Continuous Delivery/Deployment (deploy, verify, rollback).
- It helps you deliver changes faster, safer and more reliably.

2. WHY PIPELINES EXIST

- Automates repetitive manual work.
- Detects issues early.
- Improves code quality and security.
- Ensures consistent and reliable deployments.
- Reduces human error.
- Enables faster delivery and rollback when needed.
- Supports teamwork and visibility.

Faster
Safer
Higher Quality
Happier Teams

3. CI/CD PIPELINE STAGES

Stage	What it does
Code	Write and commit code
Build	Compile/build the application
Test	Run automated tests
Scan	Check for security vulnerabilities
Package	Create deployable artifact (e.g. image)
Deploy	Deploy to environment (dev/staging/prod)
Verify	Run health checks and tests
Rollback	Revert if something goes wrong

4. CI/CD PIPELINE FLOW



5. COMMON TOOLS

Category	Examples
Source Control	Git, GitHub, GitLab, Bitbucket
CI/CD Tools	Jenkins, GitHub Actions, GitLab CI, Azure DevOps
Build Tools	Maven, Gradle, npm
Test Tools	JUnit, pytest, Selenium
Security Scanning	SonarQube, Trivy, Snyk
Artifact Repositories	Docker Hub, ECR, Nexus, Artifactory
Deployment	Kubernetes, Docker, Helm, AWS, Azure, Terraform

6. EXAMPLE: SIMPLE GITHUB ACTIONS PIPELINE

```

.github/workflows/ci.yml
name: CI Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest
  
```

7. DEPLOYMENT STRATEGIES

- Recreate – simple, but downtime
- Rolling – update gradually (no downtime)
- Blue/Green – two environments, switch traffic
- Canary – release to small % of users first
- Feature Flags – enable features without redeploying

When to Use What?

- Use rolling for most applications.
- Use blue/green for critical systems.
- Use canary for high-risk changes.
- Use feature flags for gradual feature rollout.

8. ROLLBACK

- Rollback returns the application to a previous working version.
- It can be automatic (on failed checks) or manual.
- Keep your deployment process reversible.

Example (Kubernetes):

```

kubectl rollout history deployment app
kubectl rollout undo deployment app
  
```

9. VERIFICATION

After deployment, always verify:

- Application is running.
- Health checks pass.
- Logs show no errors.
- Monitoring metrics are healthy.
- Users can access the application.

Useful Commands

```

kubectl get pods    Check pod status
kubectl logs <pod>  View logs
curl -I <url>       Check HTTP response
  
```

10. BEST PRACTICES

- Keep pipelines fast and reliable.
- Automate tests and security scans.
- Use versioned, immutable artifacts.
- Store secrets securely (never in code).
- Deploy to multiple environments (dev → prod).
- Monitor and verify after deployment.
- Have a rollback plan.
- Keep pipelines as code (YAML).
- Continuously improve your pipeline.

**“Automate today,
build a better tomorrow.”**

PRACTICE TODAY
A STRONGER YOU TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



Build
Deploy
Grow

08 | Docker Fundamentals

Images vs Containers | Dockerfiles | Registries | Ports | Volumes | Networks | Logs | Lifecycle

1. WHAT IS DOCKER?

- Docker is a platform that packages applications and their dependencies into containers.
- Containers run the same way on any environment (dev, test, prod).
- It makes applications portable, consistent and easy to deploy.

Key Benefits

- Consistency
- Portability
- Faster deployments
- Isolation
- Works on any cloud or server

2. IMAGES vs CONTAINERS

Docker Image



- Read-only template
- Contains application and dependencies
- Immutable
- Built from a Dockerfile
- Can create multiple containers from one image

Docker Container



- Runnable instance of an image
- Has a writable layer
- Isolated from other containers
- Starts, stops, and can be removed
- Ephemeral (can be recreated)

Image = Blueprint | Container = Running Instance

3. DOCKERFILE BASICS

- A Dockerfile is a text file with instructions to build a Docker image.
- It defines the base image, dependencies, files, and commands.

Example Dockerfile

```
FROM ubuntu:22.04 # Base image
WORKDIR /app # Set working directory
COPY . . # Copy files
RUN apt-get update && apt-get install -y \ # Install dependencies
    python3
CMD ["python3", "app.py"] # Default command
```

4. USEFUL DOCKER COMMANDS

Command	What it does
<code>docker --version</code>	Show Docker version
<code>docker images</code>	List local images
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers
<code>docker pull <image></code>	Download an image
<code>docker run <image></code>	Run a container
<code>docker stop <id></code>	Stop a container
<code>docker start <id></code>	Start a container
<code>docker rm <id></code>	Remove a container
<code>docker rmi <image></code>	Remove an image
<code>docker logs <id></code>	View container logs
<code>docker exec -it <id> bash</code>	Access container shell

5. PORTS

- Ports allow communication between your container and the outside world.
- Use `-p` to map container ports to host ports.

Example

```
docker run -p 8080:80 nginx
```

- 8080 = host port
- 80 = container port

Now you can access the app at:
<http://localhost:8080>

6. VOLUMES

- Volumes persist data even if the container is removed.
- Used for databases, logs, and important files.
- Use `-v` to mount a volume.

Example

```
docker run -v /host/data:/app/data nginx
```

- /host/data = directory on host
- /app/data = directory in container

Data in volumes survives container restarts and deletions.

7. NETWORKS

- Networks allow containers to communicate with each other.
- Docker creates a default bridge network.
- You can also create custom networks.

Example

```
docker network create mynet
docker run --network mynet nginx
```

8. LOGS

- View container logs to troubleshoot issues.
- Use `docker logs` to see output.

Example

```
docker logs <container_id>
docker logs -f <container_id>
```

- `-f` follows logs in real time
- Useful for debugging

9. CONTAINER LIFECYCLE

Create

Start

Run

Stop

Remove

Containers are ephemeral. You can stop, start, and remove them anytime. Use images and volumes to recreate them when needed.

10. BEST PRACTICES

- ✓ Use official base images.
- ✓ Keep images small.
- ✓ Use `.dockerignore`.
- ✓ Don't store secrets in images.
- ✓ Use volumes for persistent data.
- ✓ Use meaningful names and tags.
- ✓ Clean up unused images/containers.
- ✓ Monitor container logs.
- ✓ Use health checks.
- ✓ Automate builds and deployments.

JUNIOR ENGINEER TIP



Containers are disposable, so store your data in volumes, keep your Dockerfiles clean, and always use version tags instead of latest.

COMMON MISTAKES

- ✗ Using latest tag in production.
- ✗ Not exposing the correct port.
- ✗ Storing sensitive data in images.
- ✗ Not using `.dockerignore`.
- ✗ Forgetting to clean up unused containers and images.

CONTAIN
DEPLOY
SCALE
SUCCEED

09 | Build a Dockerized Application

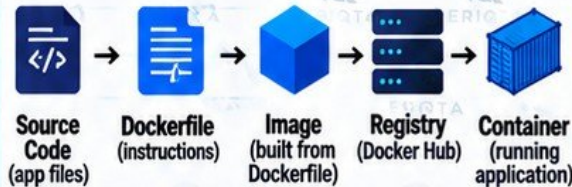
Source → Dockerfile → Image → Registry → Container

1. OVERVIEW

- Take an application from source code and run it in a Docker container.
- Use a Dockerfile to build an image.
- Store the image in a registry (Docker Hub).
- Run the image as a container.
- Learn essential Docker commands (build, run, inspect, logs, stop).
- Troubleshoot common issues.

Containers make applications portable, consistent and easy to deploy.

2. APPLICATION FLOW



Same code → Same container → Works anywhere

3. SAMPLE APPLICATION

A simple Python Flask application.

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Docker!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

4. DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

What it does:

- Uses an official Python image
- Sets working directory
- Installs dependencies
- Copies application code
- Exposes port 5000
- Runs the application

5. BUILD THE IMAGE

Build an image from the Dockerfile.

Command

docker build -t myapp:1.0

-t myapp:1.0 → image name and tag
→ build context (current directory)

Check the image:

docker images

- Lists all local images

6. RUN THE CONTAINER

Run the image as a container.

Command

docker run -d -p 5000:5000 --name myapp-container myapp:1.0

- d → run in background
- p 5000:5000 → map host port 5000 to container port 5000
- name → give the container a name

Test the application:

Open in browser:

<http://localhost:5000>

7. USEFUL COMMANDS

Command	What it does
docker ps	List running containers
docker ps -a	List all containers
docker stop <id>	Stop a container
docker start <id>	Start a container
docker rm <id>	Remove a container
docker logs <id>	View container logs
docker inspect <id>	Show detailed info
docker exec -it <id> bash	Access container shell
docker rmi <image>	Remove an image

8. PUSH TO A REGISTRY

Upload your image to Docker Hub.

- Login to Docker Hub
docker login
- Tag your image
docker tag myapp:1.0 yourusername/myapp:1.0
- Push the image
docker push yourusername/myapp:1.0

Now your image can be pulled and run on any server.

9. TROUBLESHOOTING

Common issues and fixes.

Issue	Fix
Container exits immediately	Check logs (docker logs <id>)
Port already in use	Change host port (e.g. 5001:5000)
Application not reachable	Check container is running and port mapping (docker ps)
Build fails	Check Dockerfile syntax and build context
Permission errors	Check file permissions and use correct user
Image too large	Use slim base image and .dockerignore

10. BEST PRACTICES

- Use official base images (e.g. python:slim).
- Keep images small (use .dockerignore).
- Use meaningful image and container names.
- Don't run containers as root (when possible).
- Use environment variables for configuration.
- Expose only necessary ports.
- Check logs when something goes wrong.
- Tag and push images to a registry.
- Keep your Dockerfiles clean and documented.
- Practice building and running different apps.

COMMON MISTAKES

- Using latest tag in production.
- Exposing unnecessary ports.
- Not checking container logs.
- Ignoring .dockerignore.
- Keeping sensitive data in images.



JUNIOR ENGINEER TIP

Build. Run. Break. Fix. Repeat.
The best way to learn Docker is to actually build and run real applications.

SAME SKILLS
BIGGER
OPPORTUNITIES

Learn
Build
Grow
with VERIQTA

10 | Kubernetes Fundamentals

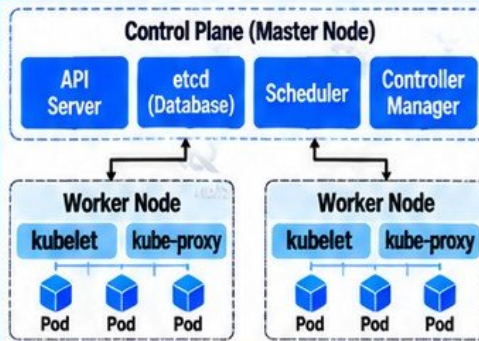
Cluster | Control Plane | Worker Nodes | Pods | Deployments | ReplicaSets | Services | Namespaces | ConfigMaps | Secrets

1. WHAT IS KUBERNETES?

- Kubernetes (K8s) is an open-source platform to deploy, manage and scale containerized applications.
- It automates scheduling, self-healing, scaling and networking.
- It runs containers across a cluster of machines (nodes).
- It works with any container runtime (e.g. containerd).

Kubernetes = Automation + High Availability + Scalability for your applications.

2. KUBERNETES ARCHITECTURE



The control plane manages the cluster. Worker nodes run your applications.

3. KEY KUBERNETES OBJECTS

Object	Purpose
Cluster	Group of machines running K8s
Control Plane	Manages the cluster
Worker Node	Runs your application workloads
Pod	Smallest deployable unit (one or more containers)
Deployment	Manages replicas of Pods
ReplicaSet	Ensures desired number of Pods
Service	Provides stable network access
Namespace	Logical isolation within a cluster
ConfigMap	Stores non-sensitive configuration
Secret	Stores sensitive data (base64 encoded)

4. USEFUL KUBECTL COMMANDS

Command	What it does
<code>kubectl version</code>	Check client and server version
<code>kubectl cluster-info</code>	Show cluster information
<code>kubectl get nodes</code>	List worker nodes
<code>kubectl get pods -A</code>	List all pods (all namespaces)
<code>kubectl get deployments</code>	List deployments
<code>kubectl get services</code>	List services
<code>kubectl get namespaces</code>	List namespaces
<code>kubectl describe <resource></code>	Show detailed information
<code>kubectl logs <pod></code>	View pod logs
<code>kubectl exec -it <pod> -- bash</code>	Access pod shell

5. PODS

- A Pod is the smallest deployable unit.
- It can run one or more containers.
- Pods are ephemeral (can be recreated).

Example Pod (pod.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```



Pod

You usually don't create pods directly in production. Use Deployments.

6. DEPLOYMENTS & REPLICASETS

- A Deployment manages the desired number of pods.
- It uses a ReplicaSet to keep pods running.
- It supports rolling updates and rollbacks.

Example Deployment (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

ReplicaSet ensures the desired number of pods are always running.



Self-healing
Always running

7. SERVICES

- A Service provides a stable network endpoint to access pods.
- It load balances traffic across healthy pods.
- Types: ClusterIP (default), NodePort, LoadBalancer, Headless.

Example Service (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```



Services remain stable even when pods are recreated.

8. NAMESPACES, CONFIGMAPS & SECRETS

- Namespaces: isolate resources (dev, test, prod).
- ConfigMaps: store non-sensitive configuration.
- Secrets: store sensitive data (passwords, keys).

Namespace Example

```
kubectl create namespace dev
kubectl get namespaces
```

ConfigMap Example

```
kubectl create configmap app-config \
--from-literal=APP_ENV=dev
```

Secret Example

```
kubectl create secret generic db-secret \
--from-literal=DB_PASSWORD=secret123
```

Use ConfigMaps for configuration and Secrets for sensitive information.

9. CONTAINER LIFECYCLE



- Image pulled from registry.
- Container created and started.
- Application runs.
- Logs can be viewed.
- Container can be stopped and removed.

10. BEST PRACTICES

- ✓ Use namespaces for environment isolation.
- ✓ Use Deployments, not individual pods.
- ✓ Keep container images small.
- ✓ Use ConfigMaps and Secrets (not hardcoded values).
- ✓ Define resource requests and limits.
- ✓ Use health checks (liveness/readiness).
- ✓ Monitor your cluster and applications.
- ✓ Practice deployments, updates and rollbacks.

PRACTICE TODAY
A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



Real Skills
Real Projects
Real Freedom

SKILLS | PROJECTS | CAREER | FREEDOM

11 | Deploy an Application to Kubernetes

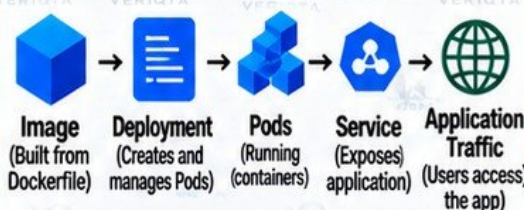
Image → Deployment → Pods → Service → Application Traffic

1. OVERVIEW

- Take a container image and deploy it to Kubernetes.
- Use a Deployment to manage pods.
- Expose the application using a Service.
- Learn essential kubectl commands to view, describe, get logs and apply manifests.
- Test the application and access it through a service.

Same process used in real production environments.

2. DEPLOYMENT FLOW



You define the desired state and Kubernetes takes care of the rest.

3. PREREQUISITES

- ✓ A Kubernetes cluster (local or cloud)
- ✓ kubectl installed and configured
- ✓ A container image (e.g. from Docker Hub or your own registry)
- ✓ Basic understanding of YAML
- ✓ A terminal / command line

You can use Minikube, Kind, or a cloud provider (EKS, GKE, AKS) for practice.

4. DEPLOYMENT YAML (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.25
          ports:
            - containerPort: 80
```

This creates 3 replicas of your application pods using the nginx image.



5. SERVICE YAML (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort # Use LoadBalancer
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80 # Service port
      targetPort: 80 # Container port
      nodePort: 30080 # (for NodePort)
```

This exposes your application and forwards traffic to the pods.

6. USEFUL KUBECTL COMMANDS

Command	What it does
<code>kubectl apply -f file.yaml</code>	Create or update resources from a YAML file
<code>kubectl get deployments</code>	List deployments
<code>kubectl get pods</code>	List running pods
<code>kubectl get services</code>	List services
<code>kubectl describe pod <pod></code>	Show detailed information
<code>kubectl logs <pod></code>	View pod logs
<code>kubectl get all</code>	Show all resources
<code>kubectl get events</code>	View recent events
<code>kubectl delete -f file.yaml</code>	Delete resources

Use `-n <namespace>` if you are working in a different namespace.

7. STEP BY STEP

- 1 Apply the Deployment
`kubectl apply -f deployment.yaml`
- 2 Check the pods
`kubectl get pods`
- 3 Apply the Service
`kubectl apply -f service.yaml`
- 4 Check the service
`kubectl get services`
- 5 Access the application
`http://<NodeIP>:30080` (for NodePort)
- 6 View logs
`kubectl logs <pod-name>`

8. VERIFY THE DEPLOYMENT

- Pods are running: `kubectl get pods`
- Deployment status: `kubectl get deployment myapp`
- Service is created: `kubectl get services`
- Check application: Open in browser or use curl (`curl http://<NodeIP>:30080`)
- View logs: `kubectl logs <pod-name>`
- Describe resource: `kubectl describe pod <pod-name>`

If something is not working, check:

- Pod status and events
- Container logs
- Service type and ports
- Network policies or firewall rules

9. COMMON ISSUES & FIXES

Issue	Fix
Pods not running	Check logs, image name, and events
Image pull error	Verify image name and registry access
Service not accessible	Check service type, ports and firewall
CrashLoopBackOff	Check container logs and configuration
Wrong image version	Use correct tag and redeploy
Application not loading	Verify service endpoint and pod health

10. BEST PRACTICES

- ✓ Use meaningful names and labels.
- ✓ Keep your YAML files in version control (Git).
- ✓ Use Deployments (not individual pods).
- ✓ Use a separate namespace for each environment.
- ✓ Use health checks (readiness/liveness probes).
- ✓ Monitor your application after deployment.
- ✓ Clean up unused resources.



JUNIOR ENGINEER TIP

Start small. Deploy a simple application like Nginx. Understand each resource. Then try a real application.

PRACTICE TODAY
A STRONGER TOMORROW

Kubernetes
Skills
Real Opportunities



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta

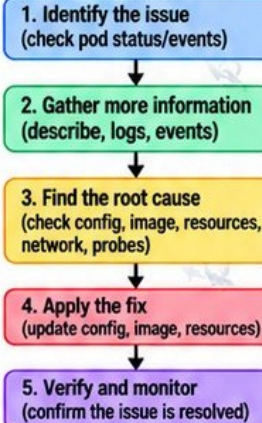
12 | Kubernetes Troubleshooting

Pending Pods | CrashLoopBackOff | ImagePullBackOff | Failed Probes | Configuration Problems | Networking Failures | Debugging Flow

1. COMMON POD PROBLEMS

Issue	Status	What it means
Pending	Pending	Pod cannot be scheduled on a node.
CrashLoopBackOff	CrashLoopBackOff	Container keeps crashing and restarting.
ImagePullBackOff	ImagePullBackOff	Cannot pull the container image.
Failed Probes	Unhealthy	Liveness or readiness probe is failing.
Config Issues	Error	Wrong or missing configuration (ConfigMap, Secret, env vars).
Networking Failures	Error	Cannot reach other services or external resources.

2. TROUBLESHOOTING FLOW



3. USEFUL KUBECTL COMMANDS

Command	What it does
<code>kubectl get pods</code>	List all pods
<code>kubectl get pods -o wide</code>	Show more details (node, IP, etc.)
<code>kubectl describe pod <pod></code>	Show detailed information
<code>kubectl logs <pod></code>	View pod logs
<code>kubectl logs <pod> -c <container></code>	View logs for a specific container
<code>kubectl get events --sort-by=.metadata.creationTimestamp</code>	View recent events
<code>kubectl get nodes</code>	Check node status
<code>kubectl get svc</code>	List services
<code>kubectl get endpoints</code>	Check service endpoints
<code>kubectl exec -it <pod> -- bash</code>	Access a running container
<code>kubectl top pods</code>	Check resource usage

4. PENDING PODS

- Often caused by insufficient resources, node selectors, taints/tolerations, or scheduling rules.
- Check events and describe output.

Example Commands

```
kubectl describe pod <pod>
kubectl get events | grep <pod>
```

Common Fixes

- Add more resources (CPU/Memory)
- Fix node selectors or tolerations
- Check if cluster nodes are ready

5. CRASHLOOPBACKOFF

- Container is starting, crashing, and restarting repeatedly.
- Check logs to find the error.
- Often caused by application errors or misconfiguration.

Example Commands

```
kubectl logs <pod>
kubectl describe pod <pod>
```

Common Fixes

- Fix application code or config
- Check environment variables
- Verify dependent services

6. IMAGEPULLBACKOFF

- Cannot pull the container image from the registry.
- Check image name, tag, and registry credentials.

Example Commands

```
kubectl describe pod <pod>
kubectl get events | grep ImagePullBackOff
```

Common Fixes

- Verify image name and tag
- Check registry credentials (imagePullSecrets)
- Ensure the image exists and is accessible

7. FAILED PROBES

- Liveness probe failing → container is restarted.
- Readiness probe failing → pod is not ready to receive traffic.
- Check probe configuration and application health.

Example Commands

```
kubectl describe pod <pod>
kubectl logs <pod>
```

Common Fixes

- Fix application endpoint (e.g. /health)
- Increase initialDelaySeconds
- Check network connectivity
- Verify the application is actually running

8. CONFIGURATION PROBLEMS

- Missing or incorrect ConfigMaps or Secrets.
- Wrong environment variables.
- Application expecting a different config.

Example Commands

```
kubectl get configmap
kubectl get secret
kubectl describe pod <pod>
```

Common Fixes

- Verify ConfigMap and Secret names
- Check mounted volumes and environment variables
- Ensure data is correctly formatted

9. NETWORKING FAILURES

- Pod cannot reach other services or external endpoints.
- Check Services, Endpoints, NetworkPolicies, and DNS.

Example Commands

```
kubectl get svc
kubectl get endpoints
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- curl <url>
```

Common Fixes

- Check Service and Endpoints
- Verify DNS resolution
- Review NetworkPolicies and firewall rules
- Ensure correct ports and protocols

10. BEST PRACTICES

- Always check events and logs first.
- Use describe to get detailed information.
- Understand the pod lifecycle and statuses.
- Keep resource requests and limits realistic.
- Use meaningful names and labels.
- Monitor probes, logs, and metrics.
- Document common issues and fixes.
- Build a troubleshooting checklist for your team.
- Practice in a real or local Kubernetes environment (Minikube, Kind, or cloud).

COMMON MISTAKES

- Ignoring events and logs.
- Using the wrong namespace.
- Misconfigured probes.
- Incorrect image names or tags.
- Not checking resource limits.
- Assuming it's a Kubernetes issue (when it's the application).



JUNIOR ENGINEER TIP

When something is not working, don't guess. Use kubectl, check events, check logs, and follow a systematic approach.

PRACTICE TODAY
A STRONGER TOMORROW



Build
Deploy
Grow

13 | Cloud Computing for DevOps

Regions | Availability Zones | Compute | Storage | Databases | Networking | IAM |
Load Balancing | Autoscaling | Shared Responsibility

1. WHAT IS CLOUD COMPUTING?

- On-demand access to computing resources over the internet.
- Pay for what you use.
- Scalable, flexible, and highly available.
- Helps DevOps teams build, deploy, and operate applications faster.
- Major providers: AWS, Microsoft Azure, Google Cloud (GCP).



AWS



Azure

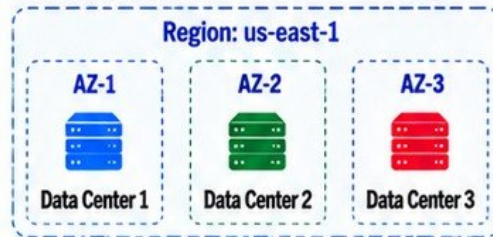


Google Cloud

Same core concepts. Different names.
Same opportunities.

2. REGIONS AND AVAILABILITY ZONES

- Region:** A geographical area (e.g., us-east-1).
- Availability Zone (AZ):** One or more isolated data centers within a region.
- Multiple AZs provide high availability and fault tolerance.



Deploy across multiple AZs to improve availability and resilience.

3. COMPUTE SERVICES

- Provide the processing power to run applications.
- Examples:



AWS EC2 (VMs)



Azure VMs



Compute Engine (GCP)



Containers (ECS/EKS, AKS, GKE)



Serverless (AWS Lambda, Azure Functions, Google Cloud Functions)

Choose the right compute service based on your workload, scalability, and cost.

4. STORAGE SERVICES

- Store application data, logs, backups, and static files.
- Types:



Object Storage (S3, Blob Storage, Cloud Storage)



Block Storage (EBS, Managed Disks, Persistent Disks)



File Storage (EFS, Azure Files, Filestore)

Use the right storage type for performance, durability, and cost.

5. DATABASE SERVICES

- Managed databases reduce operational overhead.
- Common options:



Relational (RDS, Azure SQL, Cloud SQL)



NoSQL (DynamoDB, Cosmos DB, Firestore)

Use managed databases for backup, high availability, and scaling.

6. NETWORKING

- Connect and secure your resources in the cloud.
- Key components:



VPC / Virtual Network



Subnets (Public / Private)



Security Groups / NSGs



Route Tables



Internet Gateway / NAT Gateway



VPC Peering / VPN / Direct Connect

A well-designed network improves security, performance, and reliability.

7. IDENTITY AND ACCESS MANAGEMENT (IAM)

- Control who can access what resources.
- Follow the principle of least privilege.
- Use roles instead of long-lived access keys.
- Manage users, groups, and policies.
- Integrate with your identity provider (SSO).



Right permissions. Right people. Right access. No more.

8. LOAD BALANCING

- Distribute incoming traffic across multiple instances.
- Improve availability and scalability.
- Types:



Application Load Balancer (ALB)



Network Load Balancer (NLB)



Classic Load Balancer (CLB)

Keep your applications highly available and resilient.

9. AUTOSCALING

- Automatically adjust the number of instances based on demand.
- Helps handle traffic spikes and reduces costs.
- Types:



Horizontal Scaling (add more instances)



Vertical Scaling (increase instance size)

Scale automatically. Pay only for what you use.

10. SHARED RESPONSIBILITY MODEL

- Cloud provider secures the cloud (infrastructure).
- You secure what you build in the cloud (applications, data, configurations).

Cloud Provider Responsibility (SECURITY OF THE CLOUD)

- Physical security
- Infrastructure
- Networking
- Hardware
- Virtualization
- Managed services

Your Responsibility (SECURITY IN THE CLOUD)

- Data security
- Identity and access
- Application security
- Network configuration
- OS and patching
- Monitoring and logging

Security is a shared responsibility. Understand your part.

11. BEST PRACTICES

- Design for high availability (multi-AZ).
- Use managed services where possible.
- Implement least privilege access.
- Automate infrastructure (IaC).
- Use monitoring and alerting.
- Set up logging and audit trails.
- Plan for backup and disaster recovery.
- Optimize costs (right-size resources).
- Follow the shared responsibility model.
- Review and improve regularly.

Build secure, scalable, and cost-effective cloud environments.

CLOUD SKILLS
REAL PROJECTS
REAL OPPORTUNITIES



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



VERIQTA

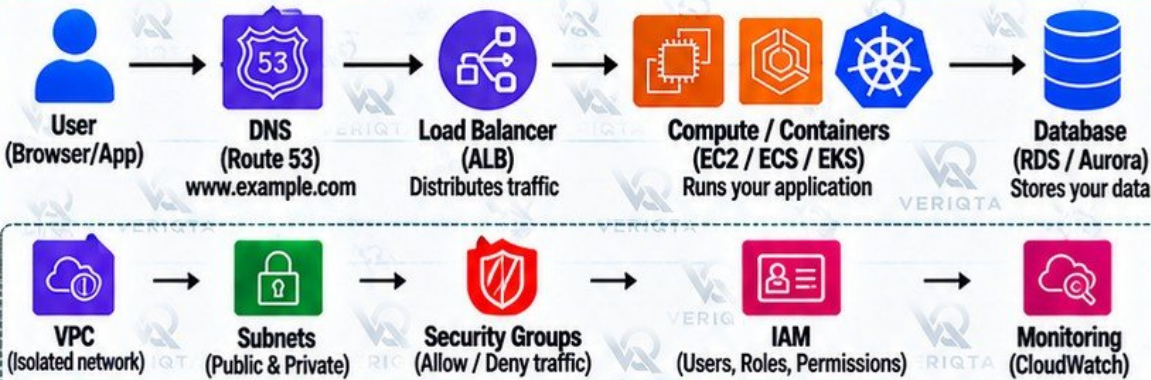
Learn
Build
Grow
with VERIQTA

SKILLS | PROJECTS | CAREER | FREEDOM

14 | AWS Application Architecture

User → DNS → Load Balancer → Compute/Containers → Database
with VPC, Subnets, Security Groups, IAM, and Monitoring

1. HIGH-LEVEL ARCHITECTURE



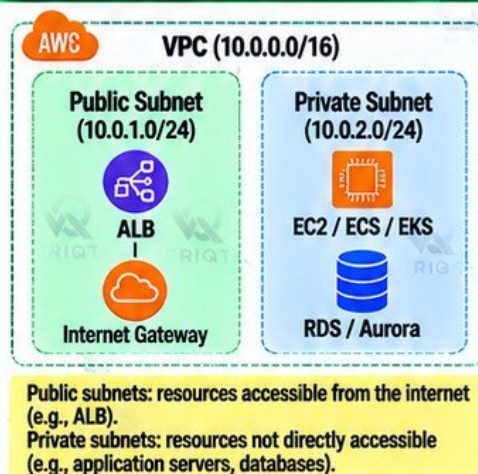
- ✓ Scalable
- ✓ Highly available
- ✓ Secure
- ✓ Cost optimized
- ✓ Production ready

Many real applications also use: S3, CloudFront, API Gateway, RDS (Aurora), ElastiCache, SNS/SQS, and more.

2. KEY AWS SERVICES

Service	Purpose
Route 53	DNS (domain name resolution)
ALB (ELB)	Distributes traffic to targets
EC2	Virtual servers (compute)
ECS / EKS	Run containers
RDS / Aurora	Managed database
VPC	Isolated network environment
Subnets	Public and private network ranges
Security Groups	Virtual firewall (allow/deny traffic)
IAM	Manage access and permissions
CloudWatch	Monitoring, logs, and metrics
S3	Object storage (static files, backups)
CloudFront	CDN (faster global delivery)

3. VPC AND SUBNETS



4. SECURITY GROUPS

- Act as a virtual firewall.
- Control inbound and outbound traffic.
- Stateful (return traffic is automatically allowed).

Example Rules (Web App)

Type	Port	Source	Purpose
HTTP	80	0.0.0.0/0	Web traffic
HTTPS	443	0.0.0.0/0	Secure traffic
SSH	22	Your IP	Admin access
Custom	3306	App SG	Database access

Use the principle of least privilege (only allow what is needed).

5. IAM (IDENTITY AND ACCESS MANAGEMENT)

- Control who can access AWS resources.
- Use users, groups, and roles.
- Attach policies to define permissions.
- Use roles for EC2, ECS, and EKS (no hardcoded keys).
- Enable MFA for sensitive accounts.

Example Policy (S3 Read Only)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

Use IAM roles instead of access keys when possible.

6. MONITORING WITH CLOUDWATCH

- Monitor resources, applications, and logs.
- Collect metrics, set alarms, and get alerts.
- View logs from EC2, ECS, EKS, and more.
- Helps you detect and troubleshoot issues.

Key Features

- Metrics (CPU, memory, requests, etc.)
- Logs (application and system logs)
- Alarms (email/SNS notifications)
- Dashboards (visualize health)
- CloudWatch Insights (query logs)

Monitoring = Faster issue detection
= Higher availability.

7. BEST PRACTICES

- ✓ Use a well-designed VPC.
- ✓ Separate public and private subnets.
- ✓ Use Security Groups (no 0.0.0.0/0 for sensitive ports).
- ✓ Use IAM roles and least privilege.
- ✓ Use managed services (RDS, ALB, etc.).
- ✓ Enable monitoring and alarms.
- ✓ Use infrastructure as code (Terraform or CloudFormation).
- ✓ Design for high availability and scalability.
- ✓ Follow the AWS Well-Architected Framework.

8. COMMON MISTAKES

- ✗ Exposing databases to the internet.
- ✗ Using overly permissive security groups.
- ✗ Not using IAM roles.
- ✗ No monitoring or alarms.
- ✗ Deploying everything in one subnet.
- ✗ Ignoring cost optimization.
- ✗ Not planning for scalability and high availability.
- ✗ Storing sensitive information in code.
- ✗ Not using infrastructure as code.

Design it right from the start to avoid costly issues later.

15 | Infrastructure as Code with Terraform

Provider → Configuration → Init → Plan → Apply → Infrastructure

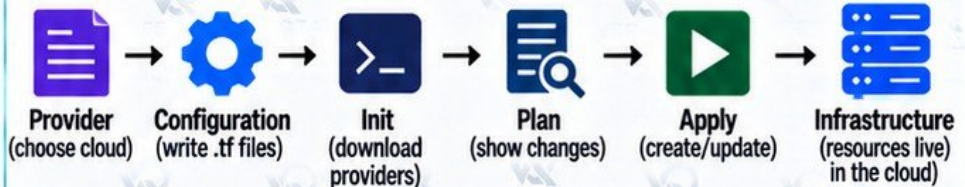
State | Modules | Variables | Outputs | Safe Changes

1. WHAT IS TERRAFORM?

- Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp.
- It lets you define, provision, and manage infrastructure using declarative configuration files (.tf).
- It works with multiple cloud providers (AWS, Azure, GCP, etc.).
- It keeps track of the current state and only makes the required changes.

Infrastructure as Code = Repeatable
+ Version Controlled + Consistent

2. HOW TERRAFORM WORKS



You describe the desired state, and Terraform makes it happen.

3. PROVIDERS

- A provider lets Terraform interact with a cloud platform (e.g., AWS, Azure, GCP).
- You define which provider to use and configure authentication.

Example (AWS Provider)

```
provider "aws" {
  region = "us-east-1"
}
```



Terraform supports many providers:
AWS, Azure, Google Cloud, Kubernetes,
Docker, and more.

4. CONFIGURATION FILES

- Configuration is written in HashiCorp Configuration Language (HCL).
- Main file: main.tf (you can split into multiple files).
- You define resources, variables, outputs, and modules.

Example (EC2 Instance)

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  tags = {
    Name = "web-server"
  }
}
```

This defines an EC2 instance in AWS.

5. CORE COMMANDS

Command	What it does
terraform init	Download providers and initialize the working directory
terraform plan	Show the changes Terraform will make
terraform apply	Create or update infrastructure
terraform destroy	Delete the infrastructure
terraform validate	Check configuration syntax
terraform fmt	Format your code
terraform show	Show details of the current state

Always run terraform plan before apply.

6. STATE MANAGEMENT

- Terraform keeps track of resources in a state file (terraform.tfstate).
- The state file maps your configuration to real infrastructure.
- It should be stored remotely (e.g., S3) for team collaboration.

Example (Remote State in S3)

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key     = "project/terraform.tfstate"
    region = "us-east-1"
  }
}
```

Keep your state file secure and never commit it to a public repository.

7. MODULES

- Modules help you reuse and organize Terraform code.
- A module is a collection of .tf files.
- Use public modules (Terraform Registry) or create your own modules.

Example (Using a Module)

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"
}
```

Modules reduce repetition and improve maintainability.

8. VARIABLES AND OUTPUTS

- Variables make your configuration flexible.
- Outputs display useful information after creation.
- Use a variables.tf file and outputs.tf file.

Example Variable	Example Output
<pre>variable "region" { description = "AWS region" type = string default = "us-east-1" }</pre>	<pre>output "instance_id" { description = "ID of the EC2 instance" value = aws_instance.web.id }</pre>

Use variables for reusable code and outputs to get important information after deployment.

9. SAFE CHANGES (BEST PRACTICES)

- ✓ Always run terraform plan before apply.
- ✓ Use version control (Git).
- ✓ Use remote state storage (e.g., S3 + DynamoDB).
- ✓ Use modules and variables.
- ✓ Follow least privilege access (IAM).
- ✓ Review changes before applying.
- ✓ Use workspaces for different environments (dev, test, prod).
- ✓ Use tagging and naming standards.

10. EXAMPLE WORKFLOW

- 1 Write or update configuration (.tf files)
- 2 Run terraform init
- 3 Run terraform plan (review changes)
- 4 Run terraform apply
- 5 Verify infrastructure in your cloud provider
- 6 Make changes and repeat

SAME SKILLS
BIGGER
OPPORTUNITIES

11. COMMON MISTAKES

- ✗ Not using a remote state backend.
- ✗ Committing state files to Git.
- ✗ Skipping terraform plan.
- ✗ Using overly broad IAM permissions.
- ✗ Hardcoding values instead of variables.
- ✗ Not using modules (copy-paste code).
- ✗ Making changes directly in the cloud console (instead of Terraform).
- ✗ Not reviewing the plan output.
- ✗ Ignoring resource dependencies.

16 | Configuration and Secrets

Environment-specific configuration | Environment variables | Kubernetes Secrets | Secret managers

Least privilege | Why credentials never belong in source code

1. WHY CONFIGURATION MATTERS

- Applications behave differently in dev, test, and production.
- Configuration makes applications flexible and portable.
- Keeps code the same, but changes behaviour per environment.
- Helps with scalability, security, and maintainability.

Same application. Different configuration. Different environments.

2. ENVIRONMENT-SPECIFIC CONFIGURATION

Development	Staging / Test	Production
• Local resources • Debug enabled • Use test data • Lower security requirements	• Similar to production • Test integrations • Use non-production data • Validate performance and security	• Real users • Real data • High availability • Strict security • Monitoring enabled

Use different configuration for each environment.
Never hardcode values in your application.

3. ENVIRONMENT VARIABLES

- Store configuration outside your code.
- Different values per environment.
- Common for database URLs, API keys, feature flags, etc.
- Easy to change without modifying code.

Example (Linux / Application)

```
export DB_HOST=prod-db.example.com
export DB_USER=appuser
export DB_PASSWORD=*****
export APP_ENV=production
```

Use environment variables for non-sensitive and sensitive configuration (with proper protection).

4. KUBERNETES SECRETS

- Store sensitive data (passwords, API keys, tokens) in Kubernetes.
- Encoded in base64 (not encrypted by default).
- Mounted as environment variables or files.
- Use RBAC to restrict access.

Create a Secret

```
kubectl create secret generic db-secret \
  --from-literal=username=appuser \
  --from-literal=password=StrongPass123
```

Use in a Pod (env variable)

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```



Kubernetes Secrets keep sensitive data out of your application code.

5. SECRET MANAGERS

- Dedicated tools to store and manage secrets securely.
- Provide encryption, access control, rotation, and audit logs.
- Common tools:

Tool	Description
AWS Secrets Manager	Store, rotate, and manage secrets in AWS
Azure Key Vault	Manage secrets, keys, and certificates in Azure
Google Secret Manager	Securely store secrets in GCP
HashiCorp Vault	Multi-cloud secret management
GitHub Secrets	Store secrets for CI/CD workflows
Kubernetes Secrets	Native Kubernetes secret storage

Use a secret manager for production environments.
Enable rotation and auditing.

6. LEAST PRIVILEGE ACCESS

- Give only the minimum access required.
- Limit who and what can access secrets.
- Use roles and policies (IAM/RBAC).
- Rotate credentials regularly.
- Monitor access and usage.

Example (AWS IAM Policy - Least Privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:app/*"
    }
  ]
}
```



Only allow the exact permissions needed.
Avoid full access.

7. WHY CREDENTIALS NEVER BELONG IN SOURCE CODE

- ✗ Can be exposed in public repositories.
- ✗ Can be accidentally shared.
- ✗ Hard to rotate if leaked.
- ✗ Attackers can misuse them.
- ✗ Violates security best practices and compliance (e.g., ISO 27001, SOC 2).

If it's in your code, it can be leaked.
Always use environment variables, Kubernetes Secrets, or a dedicated secret manager.

8. BEST PRACTICES

- ✓ Use environment-specific configuration.
- ✓ Keep secrets out of source code.
- ✓ Use a secret manager for sensitive data.
- ✓ Apply least privilege access.
- ✓ Rotate credentials regularly.
- ✓ Use encrypted storage and secure transmission.
- ✓ Audit and monitor secret access.
- ✓ Use Kubernetes Secrets or external secret operators.
- ✓ Document your configuration and secret management process.
- ✓ Follow your organization's security policies.

9. COMMON MISTAKES

- ✗ Hardcoding credentials in code.
- ✗ Committing .env files to Git.
- ✗ Using the same credentials across environments.
- ✗ Giving excessive IAM or RBAC permissions.
- ✗ Not rotating secrets.
- ✗ Storing secrets in plain text.
- ✗ Ignoring audit logs.
- ✗ Not using a secret manager in production.
- ✗ Exposing secrets in CI/CD logs.
- ✗ Relying only on Kubernetes Secrets without encryption.

Security mistakes can lead to data breaches, service downtime, and loss of trust.

PRACTICE TODAY
A STRONGER TOMORROW



@veriqta



veriqta



veriqta



veriqta



veriqta



@veriqta



Real Skills
Real Projects
Real Freedom

17 | Observability and Monitoring

Metrics | Logs | Traces | Dashboards | Alerts | Prometheus | Grafana
Application Health | Diagnosing Production Failures

1. WHAT IS OBSERVABILITY?

- Observability helps you understand what is happening in your system.
- It uses metrics, logs, and traces to give full visibility.
- Helps you detect, investigate, and fix problems quickly.
- Essential for reliable, scalable, production systems.

If you can't observe it, you can't manage it.

2. THE THREE PILLARS

Metrics



- Numeric data over time
- e.g. CPU, memory, request count
- Used for alerting and dashboards

Tells you "WHAT" is happening

Logs



- Detailed events and messages
- e.g. error logs, access logs
- Used for debugging and root cause analysis

Tells you "WHY" it happened

Traces



- Track requests across services
- Shows the full journey of a request
- Helps find latency and bottlenecks

Tells you "WHERE" it happened

3. KEY TOOLS



Prometheus
(Metrics collection and storage)



Grafana
(Dashboards and visualization)



Loki
(Log aggregation for Kubernetes)



Jaeger
(Distributed tracing)



OpenTelemetry
(Traces, metrics, logs)

4. PROMETHEUS BASICS

- Collects and stores metrics.
- Uses a pull model.
- Metrics exposed via /metrics endpoint.
- Works with Grafana for visualization.



Example metric (application):

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1027
http_requests_total{method="POST",status="500"} 3
```

Applications must expose metrics.
Common format: Prometheus metrics.

5. GRAFANA DASHBOARDS

- Visualize metrics, logs, and traces.
- Create dashboards for system and application health.
- Set up alerts.
- Use pre-built dashboards or build your own.



Good dashboards give quick visibility into the health of your systems.

6. LOGGING WITH LOKI

- Collects and stores logs efficiently.
- Integrates with Grafana.
- Common in Kubernetes environments.
- Useful for application and infrastructure logs.

Example log query (LogQL):

```
{job="app"} |= "error"
```

Use structured logs with meaningful fields (timestamp, level, service, message).

7. TRACING WITH JAEGER / OPENTELEMETRY

- Track requests across multiple services.
- Identify latency and bottlenecks.
- Understand dependencies.
- Use OpenTelemetry for instrumentation.

Request Flow (Trace)



Traces show the full journey of a request from start to finish.

8. ALERTING

- Set alerts for critical metrics (CPU, memory, errors, latency).
- Use alerting rules in Prometheus.
- Send notifications (email, Slack, etc.).

Example alert rule:

```
groups:
- name: example
  rules:
  - alert: HighErrorRate
    expr: rate(http_requests_total{status="500"}[5m]) > 0.05
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"
```

Alert on symptoms that matter to the business (e.g. user impact).

9. APPLICATION HEALTH

- Use health checks (readiness & liveness).
- Monitor key business metrics (e.g. signups, orders).
- Track SLIs and SLOs.
- Set up synthetic monitoring (external checks).

Example Kubernetes probes:

```
livenessProbe:
- httpGet:
  path: /health
  port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
```

A healthy application is responsive, reliable, and meets user needs.

10. DIAGNOSING PRODUCTION FAILURES

- 1 Check alerts and dashboards (what is impacted?)
- 2 Look at logs (find errors and patterns)
- 3 Check traces (find where it failed)
- 4 Validate infrastructure (CPU, memory, network)
- 5 Identify root cause and fix
- 6 Confirm recovery and monitor

Use metrics, logs, and traces together for faster resolution.

11. BEST PRACTICES

- ✓ Instrument your applications.
- ✓ Use meaningful metric and log labels.
- ✓ Create useful dashboards.
- ✓ Set up alerting for critical issues.
- ✓ Monitor both infrastructure and business metrics.
- ✓ Use distributed tracing for microservices.
- ✓ Keep retention and storage in mind.
- ✓ Regularly test your monitoring and alerts.
- ✓ Document your observability setup.

12. COMMON MISTAKES

- ✗ No monitoring or alerting.
- ✗ Only monitoring infrastructure, not applications.
- ✗ Too many or too few alerts.
- ✗ Ignoring logs and traces.
- ✗ Using unstructured logs.
- ✗ Not setting up dashboards.
- ✗ Not testing alerts.
- ✗ Alerting on everything (alert fatigue).
- ✗ No runbooks for incidents.

Monitoring is not a luxury. It's a requirement for production.

OBSERVABILITY
TURNS INCIDENTS
INTO INSIGHTS

18 | DevSecOps Fundamentals

Secure SDLC | Dependency Scanning | Image Scanning | Secrets Scanning | IAM | Least Privilege | Supply-chain Security | Pipeline Security

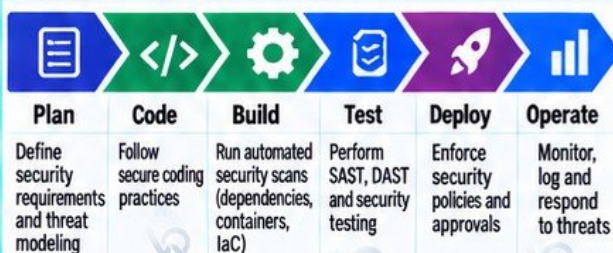
1. WHAT IS DEVSECOPS?

- Integrates security into every stage of the software development lifecycle (SDLC).
- Security is everyone's responsibility (Dev + Sec + Ops).
- Detects and fixes security issues early and automatically.
- Helps deliver secure, reliable and compliant applications faster.

"Shift Security Left"

Find and fix security issues early before they reach production.

2. SECURE SDLC



Security at every stage. Not just at the end.

3. DEPENDENCY SCANNING

- Scans open-source libraries for known vulnerabilities.
- Helps prevent vulnerable components in your application.
- Tools: Snyk, OWASP Dependency-Check, GitHub Dependabot.

Example (Snyk)

```
snyk test
snyk monitor
```



Keep your dependencies up to date and free from known vulnerabilities.

4. IMAGE SCANNING

- Scans container images for vulnerabilities, malware, and misconfigurations.
- Run scans before pushing to a registry and in your CI/CD pipeline.
- Tools: Trivy, Gype, Clair, ECR scanning.

Example (Trivy)

```
trivy image myapp:latest
```



Never deploy unscanned images to production.

5. SECRETS SCANNING

- Detects hardcoded secrets in code, configuration files, and commits.
- Prevents accidental exposure of API keys, passwords, and tokens.
- Tools: GitLeaks, TruffleHog, GitLeaks, GitHub secret scanning.

Example (GitLeaks)

```
gitleaks detect --source .
```



Secrets never belong in source code. Use environment variables or a secret manager.

6. IAM AND LEAST PRIVILEGE

- Use Identity and Access Management (IAM) to control who can access what.
- Grant only the minimum permissions required (least privilege).
- Use roles instead of long-lived access keys.
- Regularly review and rotate credentials.

Example (AWS IAM Policy - Least Privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```



Minimum access. Maximum security.

7. SUPPLY-CHAIN SECURITY

- Secure all components in your software supply chain (code, dependencies, images, artifacts).
- Verify the integrity and origin of software (artifact signing, checksums).
- Use trusted sources and private registries.
- Tools: Sigstore, Cosign, SBOM (Software and if Materials), SLSA.



Know what you are using, where it comes from, and that it has not been tampered with.

8. PIPELINE SECURITY

- Integrate security checks into your CI/CD pipeline.
- Scan code, dependencies and images automatically.
- Enforce security gates (fail the pipeline on critical issues).
- Protect pipeline secrets and use short-lived credentials.
- Tools: GitHub Actions, GitLab CI, Jenkins, Azure DevOps with security plugins.



Automate security. Make it part of your pipeline.

9. BEST PRACTICES

- Follow secure coding practices.
- Use automated security scanning.
- Keep dependencies and base images updated.
- Use secret managers (e.g., AWS Secrets Manager, HashiCorp Vault).
- Apply least privilege access.
- Sign and verify artifacts.
- Monitor and log security events.
- Respond quickly to vulnerabilities.
- Regularly review and improve your security posture.

10. COMMON MISTAKES

- Hardcoding secrets in code.
- Using outdated dependencies or images.
- Ignoring security scan results.
- Giving excessive IAM permissions.
- Skipping security in the CI/CD pipeline.
- Using untrusted third-party components.
- Not monitoring for security threats.

Security mistakes today can become major breaches tomorrow.

11. REAL-WORLD TOOLS



12. KEY TAKEAWAYS

- Security is not a phase, it's a mindset.
- Automate security checks.
- Use the right tools for each stage.
- Keep secrets out of source code.
- Follow least privilege access.
- Secure your entire supply chain.
- Make security a shared responsibility.
- Build secure systems.
- Build a better future.

Same
Knowledge
Bigger
Tomorrow

19 | Production Deployment and Incident Response

Deployment strategies | Health checks | Rollback | Monitoring | Incidents | Logs

Change verification | Root cause investigation | Recovery

1. DEPLOYMENT STRATEGIES

Different ways to deploy your application to production.

Recreate	Stop old version, start new version.
Rolling Update	Gradually replace old pods.
Blue/Green	Switch traffic from old to new.
Canary	Release to a small % of users first.
A/B Testing	Test multiple versions.

Choose a strategy based on risk, downtime tolerance, and business impact.

2. DEPLOYMENT FLOW (EXAMPLE)



Automate, test, monitor, and verify every deployment.

3. HEALTH CHECKS

- Check if the application is running and responding correctly.
- Types:
 - Liveness Probe (is it alive?)
 - Readiness Probe (is it ready?)
 - Startup Probe (is it starting?)
- HTTP, TCP, or custom checks.
- Used in Kubernetes, load balancers, and monitoring tools.

Health checks prevent traffic from going to unhealthy instances.

4. ROLLBACK STRATEGIES

- Revert to the previous working version.
- Automate rollback using your deployment tool (Kubernetes, CI/CD, etc.).
- Keep previous versions available.
- Verify the rollback was successful.

Example (Kubernetes)

```
kubectl rollout history deployment/app
kubectl rollout undo deployment/app
```

When in doubt, rollback fast and safely.

5. MONITORING IN PRODUCTION

- Monitor application health, performance, and infrastructure.
- Use metrics, logs, and traces.
- Set up dashboards and alerts.
- Tools: Prometheus, Grafana, ELK, CloudWatch, Datadog, etc.



If you can't see it, you can't fix it.

6. INCIDENTS

- An incident is any unplanned disruption that affects users or the service.
- Examples: service down, high error rate, performance issues.
- Have an incident response plan.
- Communicate clearly and quickly.
- Use an incident management tool (e.g., PagerDuty, Opsgenie, Slack).

Detect → Respond → Communicate → Investigate → Resolve → Learn.

7. LOGS AND INVESTIGATION

- Check application and system logs.
- Look for errors, stack traces, and patterns.
- Use centralized logging (e.g., ELK, Loki).
- Correlate logs with metrics and traces.

Common Log Commands

```
kubectl logs <pod-name>
kubectl logs -f <pod-name>
kubectl logs --previous <pod-name>
journalctl -u <service>
tail -f /var/log/app.log
```

Logs tell you what happened.
Use them with metrics and traces.

8. CHANGE VERIFICATION

- Verify the deployment is working as expected.
- Check application functionality.
- Validate metrics, logs, and health checks.
- Confirm no increase in errors or latency.
- Use synthetic checks and real user monitoring.

Verification Checklist

- ✓ Application is healthy
- ✓ No error spikes
- ✓ Performance is normal
- ✓ All key features working
- ✓ Business metrics are stable

A deployment is not complete until it is verified in production.

9. ROOT CAUSE INVESTIGATION

- Find the underlying cause, not just the symptom.
- Use logs, metrics, and traces.
- Check recent changes (deployments, config, infrastructure).
- Look for patterns and timelines.
- Use tools like distributed tracing (Jaeger) and log analysis.

Common Causes

- Code bugs
- Configuration changes
- Resource limits (CPU, memory, disk)
- External dependencies (DB, third-party APIs)
- Infrastructure issues
- Human error

Investigate with evidence, not assumptions.

10. RECOVERY

- Apply the fix or rollback.
- Verify the service is fully restored.
- Monitor closely after recovery.
- Communicate the resolution to stakeholders.
- Document what happened.

Recovery is not just fixing the issue.
It's ensuring stability and preventing recurrence.

11. BEST PRACTICES

- ✓ Automate deployments and rollbacks.
- ✓ Use health checks and monitoring.
- ✓ Have a clear incident response plan.
- ✓ Keep logs, metrics, and traces centralized.
- ✓ Verify changes in production.
- ✓ Perform post-incident reviews (blameless).
- ✓ Learn and improve continuously.

12. COMMON MISTAKES

- ✗ Deploying without testing.
- ✗ No rollback plan.
- ✗ Not monitoring after deployment.
- ✗ Ignoring alerts.
- ✗ Slow incident response.
- ✗ Not checking logs.
- ✗ Blaming individuals instead of finding root cause.
- ✗ Not learning from incidents.

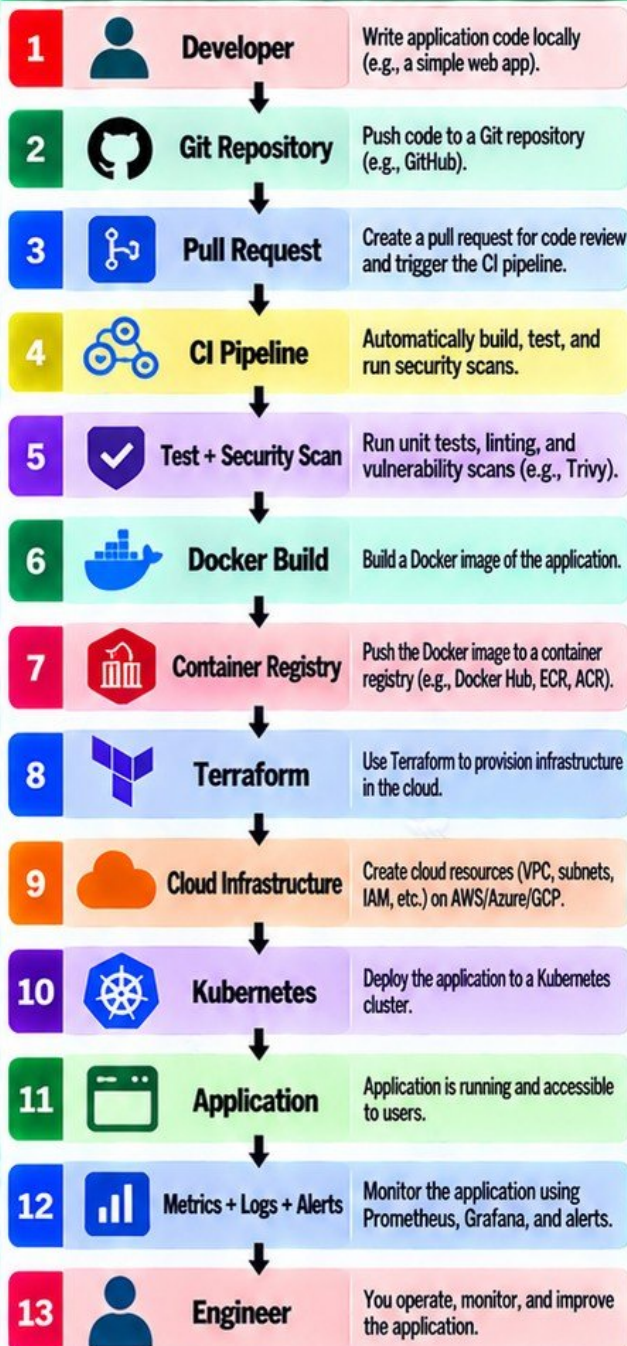
Mistakes in production are expensive.
Prepare, automate, and learn.

20 | Complete DevOps Production Project

Git → CI/CD → Docker → Registry → Terraform → Cloud → Kubernetes → Monitoring

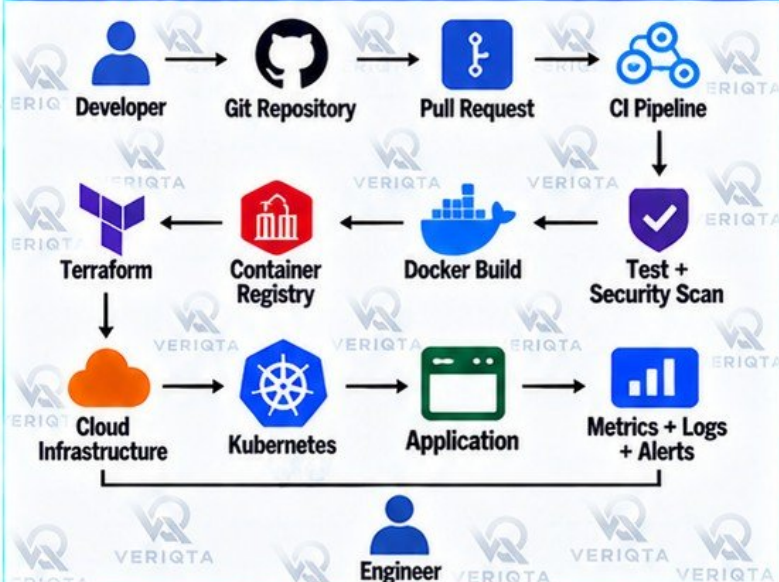
With a complete junior portfolio project

1. END-TO-END ARCHITECTURE



From code to cloud, from deployment to monitoring — a complete DevOps project.

2. ARCHITECTURE DIAGRAM



3. EXAMPLE TOOLS

Git Repository	GitHub
CI/CD	GitHub Actions, GitLab CI, Jenkins
Security Scanning	Trivy, SonarQube, Snyk
Container Registry	Docker Hub, Amazon ECR, Azure ACR
Infrastructure as Code	Terraform
Cloud	AWS, Azure, Google Cloud
Kubernetes	EKS, AKS, GKE (or local with kind/minikube)
Monitoring	Prometheus + Grafana
Logging	Loki, ELK (Elasticsearch, Logstash, Kibana)
Alerts	Grafana Alerts, Alertmanager, Slack

4. PROJECT IDEAS (JUNIOR PORTFOLIO)

- ✓ Deploy a simple web application (e.g., Nginx or Node.js).
- ✓ Set up full CI/CD with tests and security scanning.
- ✓ Use Terraform to provision cloud infrastructure.
- ✓ Deploy to Kubernetes (EKS/AKS/GKE).
- ✓ Set up monitoring and alerts.
- ✓ Document the project with a clear README.
- ✓ Include diagrams and screenshots.
- ✓ Showcase it on your GitHub portfolio.

A real project is the best proof of your skills.

5. KEY LEARNINGS

- ✓ Understand the full DevOps lifecycle.
- ✓ Gain hands-on experience with real tools.
- ✓ Learn how different tools work together.
- ✓ Build a project you can show to employers.
- ✓ Develop troubleshooting and problem-solving skills.
- ✓ Practice production-like deployments.
- ✓ Build confidence as a DevOps engineer.

Build. Document. Share.
Your portfolio opens doors.

6. COMMON MISTAKES

- ✗ Skipping tests and security scans.
- ✗ Not using infrastructure as code.
- ✗ Deploying without monitoring.
- ✗ Poor documentation.
- ✗ Not cleaning up cloud resources (costs).